



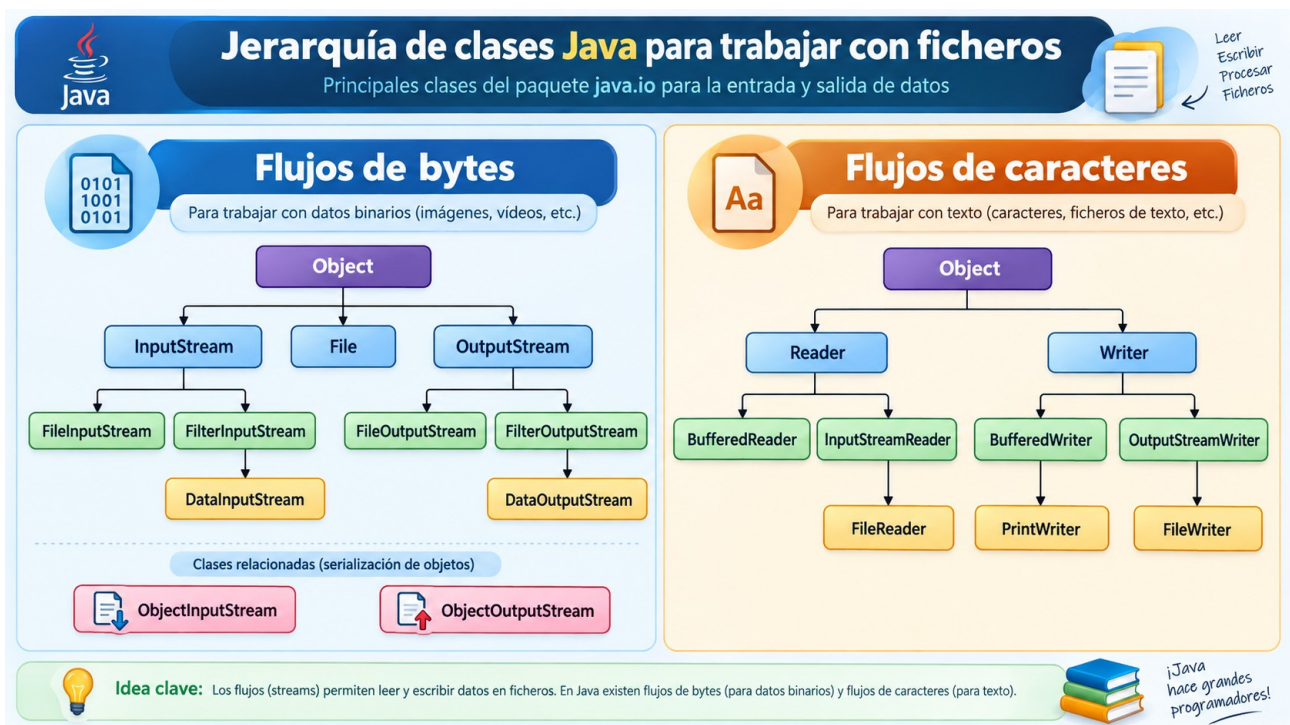
Acceso a Datos

U1: Manejo de ficheros

1. Introducción

A las operaciones, que constituyen un flujo de información del programa con el exterior, se les conoce como Entrada/Salida (E/S).

Las operaciones de E/S en Java las proporciona el paquete **java.io** que incorpora *interfaces*, *clases* y *excepciones* para acceder a todo tipo de ficheros.



Las clases de E/S de **java.io** las podemos agrupar fundamentalmente en:

- Clases para operar con ficheros en el sistema de ficheros local (File, FileFilter, ..)
- Clases para leer entradas desde un flujo de datos (InputStream, Reader,...)
- Clases para escribir entradas a un flujo de datos (OutputStream, Writer,...)
- Clases para gestionar la serialización de objetos

Algunas de estas clases las puedes ver en la figura anterior:

La documentación oficial del paquete java.io la puedes consultar en la siguiente dirección:

<http://docs.oracle.com/javase/6/docs/api/>
<http://docs.oracle.com/javase/7/docs/api/>



Acceso a Datos

U1: Manejo de ficheros

2. Clases Asociadas a la gestión de archivos y directorios

Veremos las siguientes:

- La **clase File** permite realizar la mayoría de las operaciones con ficheros y directorios(creación, eliminación, ver propiedades, listar directorios, etc).
- La **interface FilenameFilter** permite filtrar ficheros, o sea, obtener aquellos con una característica determinada, como puede ser que tengan la extensión .odt, o la que nos interese.

2.1. La clase File

La clase **File** del paquete **java.io** :

- proporciona una representación abstracta de ficheros y directorios.
- permite examinar y manipular archivos y directorios, independientemente de la plataforma en la que se esté trabajando: Linux, Windows, etc.

Las instancias de la clase **File** representan nombres de archivo, no los archivos en sí mismos.

El archivo correspondiente a un nombre puede ser que no exista, por esta razón habrá que controlar las posibles excepciones.

Un objeto de clase **File** permite examinar el nombre del archivo, descomponerlo en su rama de directorios o crear el archivo si no existe.

Constructores para crear objetos de tipo File (observa la forma de indicar la ruta en cada sistema):

- **File(String directorioyfichero):**
 - en Linux:
`File fichero1 = new File(File.separator + "home" + File.separator + "ad" + File.separator + "ut1" + File.separator + "ejemplo1.txt");`
 - en Windows:
`File fichero1 = new File("c:\\ad"+ File.separator + "ut1"+ File.separator + "ejemplo1.txt");`
- **File(String directorio, String nombrefichero)**
`String dir ="c:\\ad"+File.separator+"ut1";`
`File fichero2 = new File(dir, "ejemplo2.txt");`
- **File (File file, String fichero)**
`File file = new File(dir); //siendo dir la ruta o path del ejemplo anterior`
`File fichero3 = new File (file, "ejemplo3.txt");`

Para archivos que existen, a través del objeto **File**, un programa puede examinar los atributos del archivo, cambiar su nombre, borrarlo o cambiar sus permisos.



Acceso a Datos

U1: Manejo de ficheros

Algunos métodos importantes de la clase File son los siguientes:

Método	Descripción
<i>list()</i>	Devuelve un vector de String con los nombres de los archivos.
<i>listFiles()</i>	Devuelve un vector de objetos File.
<i>exists()</i>	Devuelve true si el fichero/directorio existe
<i>getName()</i>	Devuelve el nombre del fichero o directorio
<i>getParent()</i>	Devuelve el nombre del directorio padre o null si no existe.
<i>getAbsolutePath()</i>	Devuelve la ruta absoluta
<i>canRead()</i>	Devuelve true si el fichero se puede leer
<i>canWrite()</i>	Devuelve true si el fichero se puede escribir
<i>createNewFile()</i>	Crea un nuevo fichero, vacío, asociado a File, si y solo si no existe un fichero con dicho nombre.
<i>delete()</i>	Borra el fichero o directorio asociado al File
<i>isDirectory()</i>	Devuelve true si el objeto File corresponde a un directorio.
<i>isFile()</i>	Devuelve true si el objeto File corresponde a un fichero
<i>mkdir(); mkdirs()</i>	Crea un directorio
<i>renameTo()</i>	Renombra el objeto File
<i>length()</i>	El tamaño del fichero en bytes
<i>lastModified()</i>	Devuelve la fecha, medida en milisegundos que han transcurrido desde el 01/01/1970 hasta la última modificación.

Para indicar el directorio actual se debe utilizar “.”

Ver InformacionFichero.java, PropiedadesFichero.java, ListadoDirectorio.java

Siempre que se quiera **crear un fichero**, se puede proceder del siguiente modo:

```
Try {  
    // Creamos el objeto que encapsula el fichero  
    File fichero = new File("c:\\ad" + File.separator + "ut1" + File.separator + "miFichero.txt");  
  
    // A partir del objeto File creamos el fichero físicamente  
    if (fichero.createNewFile())  
        System.out.println("El fichero se ha creado correctamente");  
    else  
        System.out.println("El fichero no ha podido ser creado");  
}  
catch (IOException ioe) {  
    ioe.getMessage();  
}
```



Acceso a Datos

U1: Manejo de ficheros

Para **crear directorios**:

```
//Declaración de variables
String directorio = "C:\\ad"; File d = new File(directorio);
//Crear un directorio
if (d.mkdir())
    System.out.println("Directorio: " + directorio + " creado");
else
    System.out.println("El directorio: " + directorio + " no ha podido ser creado");
```

Para **borrar un fichero o directorio** se utiliza el método **delete()**. Por ejemplo:

```
if(f2.delete())
    System.out.println("Fichero borrado...");
else
    System.out.println("No se ha podido borrar el fichero");
```

Para **borrar un directorio** con Java, tendremos que borrar cada uno de los ficheros y directorios que éste contenga. Al poder almacenar otros directorios, se podría recorrer recursivamente el directorio para ir borrando todos los ficheros. Se puede listar el contenido del directorio e ir borrando con:
File[] ficheros = directorio.listFiles();

Recuerda que sabemos si el elemento es un directorio mediante ***isDirectory()***

Ver CreacionDirectorio.java y BorraDirectorio.java

2.2. Interface FilenameFilter

Hemos visto como obtener la lista de ficheros de una carpeta o directorio. A veces, nos interesa ver no la lista completa, sino los archivos que encajan con un determinado criterio.

Por ejemplo, nos puede interesar un filtro para ver los ficheros modificados después de una fecha, o los que tienen un tamaño mayor del que el que indiquemos, etc.

El interface **FilenameFilter** se puede usar para crear filtros que establezcan criterios de filtrado relativos al nombre de los ficheros. Una clase que lo implemente debe definir e implementar el método:

boolean accept(File dir, String nombre)



Acceso a Datos

U1: Manejo de ficheros

Este método devolverá *true* en el caso de que el fichero cuyo nombre se indica en el parámetro *nombre* aparezca en la lista de los ficheros del directorio indicado por el parámetro *dir*.

EJEMPLO 5. En el siguiente ejemplo vemos cómo se listan los ficheros de la carpeta `c:\ad\ud2` que tengan la extensión `.java`. Usamos *try* y *catch* para capturar las posibles excepciones, como que no exista dicha carpeta. [Filtrar.java]

```
import java.io.File;
import java.io FilenameFilter;

public class Filtrar implements FilenameFilter
{ String extension;
  // Constructor
  Filtrar(String extension){
    this.extension = extension;
  }

  public boolean accept(File dir, String name){
    return name.endsWith(extension); //endsWith(sr) devuelve true si la cadena finaliza en el valor de sr
  }

  public static void main(String[] args) {
    try {
      // Obtendremos el listado de los archivos de ese directorio
      File fichero=new File("c:\\ad\\ut1\\");
      String[] listadeArchivos = fichero.list();

      // Filtraremos por los de extension .java
      listadeArchivos = fichero.list(new Filtrar(".java"));

      // Comprobamos el número de archivos en el listado
      int numarchivos = listadeArchivos.length ;

      // Si no hay ninguno lo avisamos por consola
      if (numarchivos < 1)
        System.out.println("No hay archivos que listar");
      // Y si hay, escribimos su nombre por consola.
      else
      {
        for(int conta = 0; conta < listadeArchivos.length; conta++)
          System.out.println(listadeArchivos[conta]);
      }
    }
    catch (Exception ex) {
      System.out.println("Error al buscar en la ruta indicada");
    }
  }
}
```

De igual forma, el interface **FileFilter** se puede usar para crear filtros que establezcan criterios de filtrado relativos a objetos `File`. Se puede usar una instancia de `FileFilter` con el método ***listFiles(FileFilter filtro)*** de la clase `File` para filtrar objetos `File`.

El método **`boolean accept(File pathname)`** es el que establece el test de filtrado, y es el que debe sobrescribir la clase que implemente a `FileFilter`.

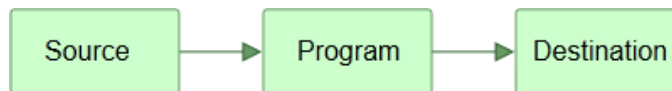


Acceso a Datos

U1: Manejo de ficheros

3. Flujos o Streams. Tipos

El siguiente esquema ilustra el principio básico de un programa que **lee datos de un origen y los escribe en un destino**.



Como ejemplos típicos en Java, de **origen y destino** podemos citar: ficheros o archivos, conexiones de red, buffers de memoria y tuberías o pipes.

En Java, un programa que necesite leer datos de un origen o escribir datos en un destino, lo hace a través de un flujo o stream.

Flujo o stream es una abstracción para tratar la comunicación de información entre un origen y un destino. La vinculación de este flujo al dispositivo físico la hace el sistema de entrada y salida de Java.

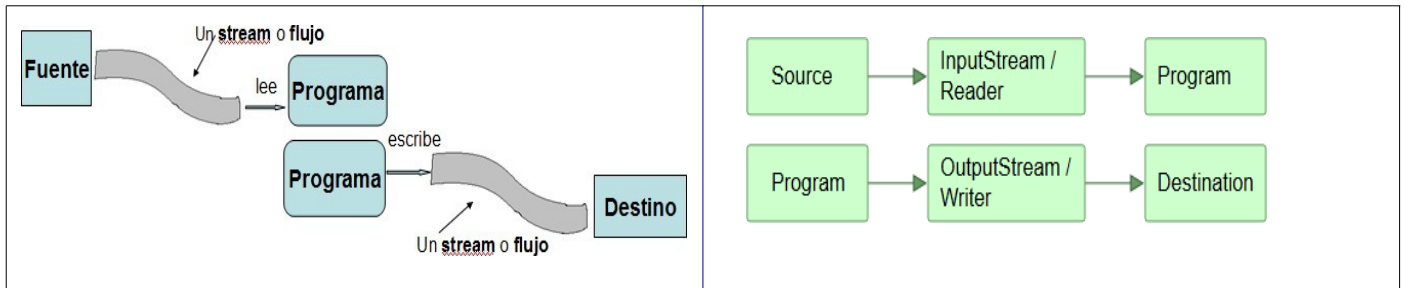
Los **tipos de flujos** definidos en **java.io** son:

- **Flujos de bytes(8 bits).** Reciben y envían datos en formato binario.
 - Lectura/Escritura de datos binarios.
 - Para el tratamiento de los flujos de bytes Java tiene dos clases abstractas: **InputStream** y **OutputStream**
 - Estas dos clases definen los métodos que sus subclases tendrán implementados y, de entre todos, destacan **read()** y **write()** que leen y escriben bytes de datos respectivamente.
- **Flujos de caracteres (16 bits).** Reciben y envían datos como secuencia de caracteres.
 - Lectura/Escritura de caracteres.
 - Para el tratamiento de los flujos de caracteres Java tiene dos clases abstractas: **Reader** y **Writer**.
 - Dichas clases manejan flujos de caracteres Unicode. Y también de ellas derivan subclases concretas que implementan los métodos definidos en ellas siendo los más destacados los métodos **read()** y **write()** que leen y escriben caracteres de datos respectivamente.



Acceso a Datos

U1: Manejo de ficheros



En java, existen dos tipos de archivos:

- **Archivos Binarios**, basados en flujos de bytes. Se crean y manejan usando flujos de bytes.
- **Archivos de Caracteres**, basados en flujos de caracteres. Se crean y manejan usando flujos de caracteres.

Las clases fundamentales para el **manejo de archivos Binarios** son:

- **FileInputStream** para lectura
- **FileOutputStream** para escritura

Junto a ambas se pueden utilizar, entre otras, las siguientes clases:

- **ObjectInputStream / ObjectOutputStream** para serialización de objetos que implementen la interfaz **Serializable**, y poder así persistir esos objetos.
- **DataInputStream/DataOutputStream** para realizar lectura/escritura de datos primitivos como : int, long, float, double, etc

Las clases fundamentales para el **manejo de archivos de Caracteres** son:

- **FileReader** para lectura
- **FileWriter** para escritura

Junto a ambas se pueden utilizar, entre otras, las siguientes clases:

- **CharArrayReader y CharArrayWriter**. para acceso a caracteres: leen y escriben un flujo de caracteres en un array de caracteres.

Además van a ser interesantes las **clases que proporcionan buferización** como:

- **BufferedInputStream, BufferedOutputStream, BufferedReader y BufferedWriter** que añaden un **buffer** intermedio, minimizando los accesos a disco, y consiguiendo que las operaciones de lectura/escritura se realicen de manera más eficiente.

Si se usan sólo **FileInputStream, FileOuputStream, FileReader o FileWriter**, cada vez que se efectúa una lectura o escritura, se hace físicamente en el disco duro. Si se leen o escriben pocos caracteres cada vez, el proceso se hace costoso y lento por los muchos accesos a disco duro.



Acceso a Datos

U1: Manejo de ficheros

Las clases **BufferedReader**, **BufferedInputStream**, **BufferedWriter** y **BufferedOutputStream** añaden un **buffer** intermedio. Cuando se lee o escribe, esta clase controla los accesos a disco. Así, si vamos escribiendo, se guardarán los datos hasta que haya bastantes datos como para hacer una escritura eficiente.

Al leer, la clase leerá más datos de los que se hayan pedido. En las siguientes lecturas nos dará lo que tiene almacenado, hasta que necesite leer otra vez físicamente. Esta forma de trabajar hace los accesos a disco más eficientes y el programa se ejecuta más rápido.

4. Formas de acceso a un fichero

La **forma de acceso a un fichero** determina la manera en la que se puede acceder a la información contenida en él, tanto para lectura como para escritura; si se puede por ejemplo localizar de forma directa cierto registro o dato, o por el contrario es necesario leer desde el principio en el fichero hasta llegar al registro o dato buscado.

Los tipos de acceso que soporta Java son:

- **Acceso secuencial**

En este caso los datos se leen de manera secuencial, desde el comienzo del archivo hasta el final. Para acceder a un dato concreto hay que leer los anteriores. Si se añade, será al final del fichero, a partir del último dato escrito. No es posible hacer inserciones entre datos escritos.

Este tipo de acceso es soportado mediante las clases:

- `FileInputStream/FileOutputStream`
- `FileReader/FileWriter`.

- **Acceso aleatorio**

Permite acceder a los datos directamente, sin necesidad de leer los anteriores y además en cualquier orden. Esto implica que el archivo debe estar disponible en su totalidad al momento de ser accedido. Se manipulan posiciones relativas (número de byte) en vez de direcciones absolutas (pista, sector), lo que hace al programa independiente de la dirección absoluta del fichero en el disco. Este tipo de acceso es soportado mediante la clase **RandomAccessFile**.



Acceso a Datos

U1: Manejo de ficheros

5. Clases para la gestión de flujos de datos

En este apartado veremos ejemplos de las diferentes operaciones que se pueden realizar sobre archivos binarios y de texto. Para ello, utilizaremos las clases apropiadas de Java, tanto para acceso secuencial como para acceso aleatorio.

5.1. Ficheros de texto

Las clases *FileReader* y *FileWriter* están optimizadas para trabajar con caracteres y con texto en general, debido a que tienen en cuenta que cada carácter Unicode está representado por dos bytes.

- Cuando trabajamos con ficheros, cada vez que leemos o escribimos en uno, debemos hacerlo dentro de un manejador de excepciones **try_catch** o mediante **throws**.
- Al usar la clase **FileReader**, se puede generar la excepción *FileNotFoundException* (fichero inexistente o no válido)
- Al usar la clase **FileWriter**, se puede generar la excepción *IOException* (disco lleno o protegido contra escritura).

Los siguientes **métodos de FileReader**, para lectura devuelven el número de caracteres leídos o -1 si es fin de fichero.

- *int read()*: lee un carácter y lo devuelve.
- *int read(char[] buf)*: lee hasta *buf.length* caracteres. Los caracteres leídos del fichero, se van almacenando en *buf*.
- *int read(char[] caracteres, int desplazamiento, int cantidad)*: lee hasta la cantidad de caracteres indicados por el parámetro cantidad y los almacena en el array caracteres comenzando desde la posición indicada por desplazamiento.

Para leer, por ejemplo, solo los primeros 20 caracteres escribimos:

```
char b[]=newchar[20];  
fr.read(b);  
System.out.println(b);
```

Ver LeerFichTexto.java



Acceso a Datos

U1: Manejo de ficheros

Algunos de los **métodos de FileWriter** para escritura de caracteres son:

- `void write(int c)`: escribe un carácter.
- `void write(char[] buf)`: escribe un array de caracteres
- `void write (char[] buf, int desplazamiento, int n)`: escribe *n* caracteres de datos del array *buf*, comenzando en *buf[desplazamiento]*.
- `void write(String str)`: escribe una cadena de caracteres.

```
String prov[]={ "Almería", "Granada", "Cadiz", "Málaga", "Córdoba", "Sevilla",  
"Huelva"};  
for (int i=0; i<prov.length; i++)  
    fw.write(prov[i]);
```

- `append(char c)`: añade un carácter a un fichero.

Estos métodos pueden lanzar la excepción **IOException**.

[Ver EscribirFichTexto.java](#)

En todos los casos anteriores, si el fichero existe se sobrescribirá, perdiendo su contenido anterior. Si queremos evitar la sobrescritura y añadir caracteres al final, utilizaremos **FileWriter** de la forma:

```
FileWriter fw=new FileWriter(fichero,true);
```

5.1.1. BufferedReader

FileReader no contiene métodos para leer líneas completas, pero **BufferedReader** sí, con el método `readLine()` que devuelve la línea leída o *null* si no hay nada o se llega al final. Soporta el método `read()` lee un carácter.

Para construir un **BufferedReader** necesitamos la clase **FileReader**.

[Ver LeerFichTextoBuff.java](#)



Acceso a Datos

U1: Manejo de ficheros

5.1.2. **BufferedWriter**

Deriva de **Writer** y añade un buffer para realizar una escritura eficiente de caracteres. Para construir un **BufferedWriter** necesitamos la clase **FileWriter**.

[Ver LeerFichTextoBuff.java](#)

5.1.3. **PrintWriter**

Posee los métodos **print(String cadena)** y **println(String cadena)** (idénticos a System.out) para escribir en un fichero. Para poder usar esta clase, primero debe crearse el flujo hacia el fichero sobre el que se desea escribir:

```
PrintWriter pw=new PrintWriter(new File("C:" +File.separator + "tmp" + File.separator +
"FichTextoPrintWriter.txt"));

for(int i=1; i<11;i++)

    pw.println("Fila número:" + i);

pw.close();
```

[Ver EscribirFichTextoPrintWriter.java](#)

5.2. Ficheros binarios

Los **Fichero binarios** almacenan secuencias de dígitos binarios que no son legibles directamente desde un editor de texto, como ocurría con los ficheros de texto.

- Dos clases que permiten trabajar con ellos son **FileInputStream** y **FileOutputStream**, para lectura y escritura respectivamente.
- Al usar la clase **FileInputStream** se puede generar la excepción **FileNotFoundException** (fichero inexistente o no válido), y al usar **FileOutputStream**, se puede generar la excepción **IOException** (disco lleno o protegido contra escritura).



Acceso a Datos

U1: Manejo de ficheros

Los **métodos para lectura** que proporciona **FileInputStream** devuelven el número de bytes leídos o -1 si se ha llegado al final del fichero. Entre ellos están:

- *int read(): lee un byte y lo devuelve*
- *int read(byte[] b): lee hasta b.length bytes de datos de una matriz de bytes*
- *int read(byte[] datos, int desplazamiento, int cantidad): lee la cantidad de bytes indicados por el parámetro cantidad y los almacena en el array datos comenzando desde la posición indicada por desplazamiento.*

Entre los **métodos para escritura** que proporciona **FileOutputStream** destacamos los siguientes:

- *void write(int b): escribe un byte*
- *void write(byte [] b) escribe un array de b bytes.*
- *void write(byte[] datos, int desplazamiento, int cantidad): Escribe hasta la cantidad de bytes indicados por el parámetro cantidad comenzando en la posición desplazamiento del array de datos.*

Por defecto, **la escritura sobrescribe el contenido del fichero**. Para evitarlo se debe crear el flujo de la siguiente forma:

```
FileOutputStream fileout = new FileOutputStream(fichero, true)
```

Ver EscribirLeerFicheroBytes.java, ContarBytesFichero.java, CopiaFicherosBytes.java

Por último, al igual que sobre los flujos de caracteres, sobre los flujos de bytes también se pueden crear **Buffers**, usando las clases **BufferedInputStream** y **BufferedOutputStream**.

```
BufferedInputStream entrada = new BufferedInputStream(new FileInputStream(fichero))
```



Acceso a Datos

U1: Manejo de ficheros

5.2.1. Datos de tipo primitivo en ficheros binarios

Las clases ***DataInputStream*** y ***DataOutputStream*** se usan para leer y escribir datos de tipos primitivos (int, float, long, etc). Estas clase definen diversos métodos **readXXX()** y **writeXXX()** para la lectura y la escritura de datos de tipo primitivo sobre un fichero.

Algunos **métodos de DataInputStream** son: (leen y devuelven un dato del tipo indicado)

- *int readInt(), char readChar(), long readLong(), float readFloat(), double readDouble(), String readUTF()*

Algunos **métodos de DataOutputStream** son: (escriben datos del tipo indicado)

- *void writeInt(int v)*
- *void writeChar(int v)*
- *void writeFloat(float v)*
- *void writeLong(long v)*
- *void writeUTF(String str)*
- *void writeChars(String str)*

Ver EscribirFicheroDatos.java, LeerFicheroDatos.java

5.2.2. Serialización de objetos

¿Cómo guardar un objeto completo, por ejemplo un objeto empleado, en un fichero?

Java permite guardar objetos en ficheros binarios mediante la **serialización**, que consiste en convertir el objeto en una secuencia de bytes que posteriormente puede ser restaurada al objeto original.

- El objeto debe implementar la **interface Serializable** que dispone de métodos para guardar y leer objetos en un fichero binario.
- Se deben usar las clases **ObjectInputStream** y **ObjectOutputStream** permiten leer y escribir objetos serializables mediante los métodos:
 - ***Object readObject()***: Usado para leer un objeto del fichero. Puede lanzar las excepciones: ***IOException, ClassNotFoundException***
 - ***void writeObject(Object obj)***: Usado para escribir el objeto especificado en el fichero. Puede lanzar la excepción ***IOException***.



Acceso a Datos

U1: Manejo de ficheros

ObjectOutputStream puede darnos algunos problemas, por ejemplo:

- si una vez que hemos escrito datos en el fichero lo cerramos, y después volvemos a abrirlo para añadirle datos (*FileOutputStream(fichero, true)*), entonces se añade una cabecera entre los objetos introducidos anteriormente y los nuevos. Esto origina el problema de que al leer el fichero se produzca una excepción.

Para resolver este problema es necesario eliminar esa cabecera intermedia. Esto se puede conseguir sobrescribiendo el método *writeStreamHeader()* de la clase *FileOutputStream*, que es el encargado de escribir las cabeceras.

Ver EscribirFicheroObjetos.java, LeerFicheroObjetos.java

5.3. Ficheros de acceso aleatorio

Todo lo que se ha visto hasta el momento son operaciones realizadas sobre ficheros a los que se accede de forma secuencial. Se empieza la lectura en el primer byte, en el primer carácter o en el primer objeto, y seguidamente se leen los siguientes uno a continuación de otro hasta llegar al final del fichero. Igualmente, cuando escribimos los datos en el fichero estos se van escribiendo a continuación de la última información escrita.

Java dispone de la clase [RandomAccessFile](#) que ofrece métodos para acceder a una posición concreta del mismo.

Este tipo de ficheros maneja un puntero que indica la posición actual en el fichero. Al crear el fichero se coloca en el byte 0, apuntado al principio del mismo. Las sucesivas lecturas y escrituras, ajustan el puntero según la cantidad de bytes leídos o escritos.

Esta clase **no forma parte de la jerarquía *InputStream/OutputStream***, ya que su comportamiento es totalmente distinto, pudiendo avanzar y retroceder dentro de un fichero.

La **creación de un fichero de acceso aleatorio** o directo se puede hacer de las siguientes formas:

- Escribiendo el nombre del fichero:

```
RandomAccessFile fichero= new RandomAccessFile(String nombre, String modoAcceso)
```

- Con un objeto *File*:

```
RandomAccessFile fichero= new RandomAccessFile(File fich, String modoAcceso)
```



Acceso a Datos

U1: Manejo de ficheros

En ambos casos *modoAcceso* puede ser:

- **r**: Abre el fichero en modo de solo lectura. El fichero debe existir. Una operación de escritura sobre este fichero lanzará la excepción *IOException*.
- **rw**: Abre el fichero en modo lectura y escritura. Si el fichero no existe lo crea.

Para **lectura y escritura en el fichero de acceso aleatorio** pueden usarse los métodos `read()` y `write()` de las clases *DataInputStream* y *DataOutputStream*.

Para **manejar el puntero o apuntador asociado a un objeto *RandomAccessFile*** existen diferentes métodos, entre ellos destacamos los siguientes:

- `long getFilePointer()`: devuelve la posición actual del puntero del fichero.
- `void seek(long position)`: coloca el puntero del fichero en una *posición* determinada, desde el comienzo del mismo.
- `long length()`: devuelve el tamaño del fichero en bytes. La posición `length()` marca el final del fichero.
- `int skipBytes(int desplazamiento)`: desplaza el puntero desde la posición actual el número de bytes indicados en desplazamiento.

5.3.1 Operaciones sobre ficheros de acceso aleatorio.

Escritura

Paquete *ficherosaleatorios*: *EscribirFicheroAleatorio*

En el ejemplo indicado, se insertan datos de empleados en un fichero aleatorio. Los datos a insertar son: identificador (valor mayor que 0), apellido, numero de departamento y salario. La longitud del registro de cada empleado es la misma (34 bytes) y los tipos que se insertan y su tamaño en bytes son:

- Identificador, entero que ocupa 4 bytes.
- Apellido, cadena de 10 caracteres. Como Java utiliza caracteres UNICODE, cada carácter de una cadena ocupa 16 bits (2 bytes). Por tanto, apellido ocupa 20 bytes.
- Departamento, tipo entero corto (short) que ocupa 2 bytes.
- Salario, tipo double, ocupa 8 bytes.



Acceso a Datos

U1: Manejo de ficheros

Lectura

Para realizar la lectura del fichero completo, se deben ir leyendo los registros en el mismo orden en el que fueron escritos. En este caso no es necesario posicionarse porque se hace un recorrido secuencial byte a byte. La lectura finalizará cuando la posición actual del fichero sea igual al tamaño del fichero:

```
file.getFilePointer()==file.length()
```

Además, solo se visualizarán los registros cuyo identificador sea mayor que 0, dado que si es 0, quiere decir que se trata de un hueco.

Paquete ficherosaleatorios: LeerTodosRegistros

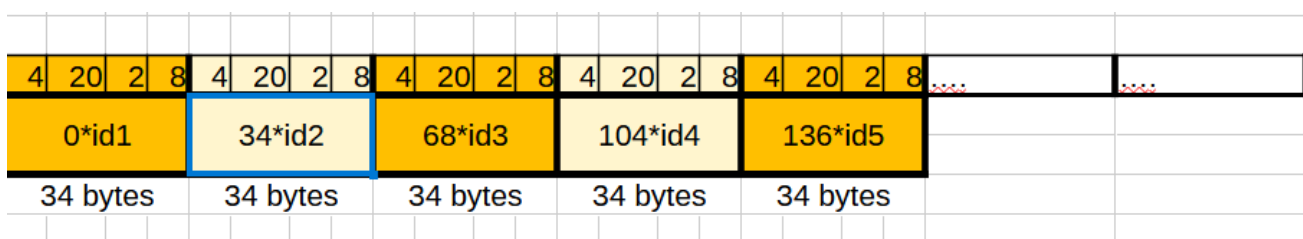
Búsqueda/Visualización

También sobre este tipo de ficheros se pueden hacer operaciones de búsqueda/visualización de un registro concreto. En ese caso es necesario pasar como parámetro el identificador del registro a buscar/visualizar. A partir del identificador se calcula la posición de inicio del registro, y si esta es mayor o igual a la longitud del fichero, querrá decir que el registro no está. En caso contrario, el registro existe y por tanto, se deberá posicionar el puntero para proceder a la lectura del registro.

Paquete ficherosaleatorios: VisualizarUnRegistro

Añadir

Para añadir registros, simplemente se debe posicionar el puntero del fichero al final del mismo, y se añade el nuevo registro:



```
long posicion=file.length();  
  
file.seek(posicion);
```



Acceso a Datos

U1: Manejo de ficheros

Actualizar

Para modificar un registro determinado, se accede a su posición y se llevan a cabo las modificaciones correspondientes del o de los campos deseados.

Por ejemplo, si se quiere modificar el departamento a 40 e incrementar en 300 el salario del registro cuyo identificador es 4, se deben seguir los siguientes pasos:

1. Se localiza del registro con el identificador determinado.
2. Se localiza dentro de dicho registro la posición del departamento, posicionándose para asignar en esa posición el nuevo valor.
3. Una vez actualizado el departamento, de nuevo debe localizarse dentro del registro la posición del salario, posicionándose para leer el valor actual del salario, se retrocede para volver a la posición inicial del salario, y finalmente, se actualiza el valor del salario.

Paquete ficherosaleatorios: ModificarEliminarRegistro

Borrar

Para el borrado de registros, estos no se pueden borrar físicamente, se debe hacer un borrado lógico. Para ello se marcará el registro a eliminar poniendo en el identificador un valor no válido, por ejemplo, 0. También se puede hacer añadiendo a los registros un campo lógico que tendrá valor 1 si el registro está activo o 0 si no está activo.

Paquete ficherosaleatorios: ModificarEliminarRegistro